



pythonTM

Chapitre 4 : Les fonctions et modules



Les fonctions :

Une fonction est une **série d'instructions regroupées sous un nom**, qui permet d'effectuer un traitement (suite d'instructions) **et** qui renvoie une valeur.

Les fonctions sont utilisées pour éviter la redondance de code en permettant d'écrire une suite d'instructions une seule fois et de l'appeler à plusieurs endroits dans le programme.

Les fonctions permettent de gagner du temps et surtout de rendre un programme plus lisible.



on (en programmation)



- prend des données en entrée – ses **paramètres**, appelés aussi **arguments** – ce sont des valeurs
- produit une valeur – son **résultat** – on dit aussi renvoie un résultat

Exemple introductif



Imaginons qu'au cours d'une application, nous avons – **plusieurs fois** – besoin de calculer la somme $s=1+2...n$. Mais à chaque fois, une somme différente.

Exemple

$S1=0$

for i in range (11):

$S1=S1+i$

$S2=0$

for i in range (81):

$S2=S2+i$

$S3=0$

for i in range (101):

$S3=S3+i$

$S4=0$

for i in range (5001):

$S4=S4+i$

D'abord l'utilisation des fonctions



Exemple

```
S1=0
for i in range (11):
    S1=S1+i
```

```
S2=0
for i in range (81):
    S2=S2+i
```

```
S3=0
for i in range (101):
    S3=S3+i
```

```
S4=0
for i in range (5001):
    S4=S4+i
```

```
def somme(n):
    S=0
    for i in range (n+1):
        S=S+i
    return S
```

```
S1=somme(10)
S2=somme(80)
S3= somme(100)
S4= somme(5000)
```



La syntaxe générale de la définition d'une fonction est la suivante :

```
def nom_de_la_fonction(p1,p2,,,,):  
    Bloc d'instructions  
    return resultat (optionnel)
```

l'indentation de ce bloc d'instructions (qu'on appelle le corps de la fonction) est obligatoire.

Le bloc instructions effectue les calculs pour réaliser les tâches confiées à la fonction définie



Paramètres

1. Sans paramètres

```
def bonjour( ):
```

```
    print(' bonjour tout le monde')
```

```
def table7( ):
```

```
    'Cette fonction affiche la table de 7'
```

```
    for n in range(1,11):
```

```
        print(n * 7, end = ' ')
```



Paramètres

2. Avec un paramètre

```
def table( b):  
    for n in range(1,11):  
        print(n * b, end = '')
```

```
def VAS( x):  
    if x >= 0 :  
        return x  
    else:  
        return -x
```





Paramètres

3. Avec plusieurs paramètres

```
def Somme (a,b ):  
    return a+b
```

```
def max(a,b ):  
    if a>b :  
        return a  
    else:  
        return b
```





Fonction avec un paramètre par défaut :

```
def saluer(nom="ami"):  
    print(f"Bonjour {nom} !")
```

```
saluer("Fatou")  
saluer()
```

```
# Affiche : Bonjour Fatou !  
# Affiche : Bonjour ami !
```



Appel d'une fonction (ou utilisation)

Appel : **nom_fonction**(argument1, argument2, ...)

Exemples

```
z = somme (2, 3)  
bonjour ()
```

```
# Appel avec deux arguments  
# Appel sans argument
```

Remarque

L'appel d'une fonction peut se faire :

- dans le **programme principal** (en dehors des fonctions),
- ou à l'intérieur d'une autre fonction.



Fonction réursive

Les fonctions dites réursives sont des fonctions qui font appel à elles-mêmes.

Pour qu'elle fonctionne correctement, elle doit toujours avoir :

- ✓ un **cas de base** (ou **cas limite**) pour arreter la **réursivité** ;
- ↻ un **appel réursif** qui rapproche la fonction du cas de base.

Exemple : Calcul de la factorielle

```
def factorielle(N):  
    if N == 0:  
        return 1  
    else :  
        return N*factorielle(N - 1)
```



Exemples

1. Ecrire une **fonction** qui retourne le produit de deux entiers
2. Ecrire une **fonction récursive** qui retourne le produit de deux entiers
3. Ecrire une **fonction** qui retourne la somme $1+2+3+\dots+n$
4. Ecrire une **fonction récursive** qui retourne la somme $1+2+3+\dots+n$



Variables locales, variables globales

On peut manipuler 2 types de variables : des variables locales et des variables globales. Elles se distinguent par ce qu'on appelle leur portée (leur "espace de visibilité", leur "durée de vie")

- Une variable définie à l'intérieur d'une fonction est une **variable locale**, elle n'est connue qu'à l'intérieur de cette fonction.
- Une variable définie à l'extérieur des fonctions est une **variable globale**. Elle est définie durant toute l'application et peut être utilisée par les différentes fonctions du programme.



Variables locales, variables globales

P1	P2
<pre>y=10 def f (a): x=1 return a+x print(y) print(f(16)) print(x)</pre>	<pre>x=10 def f (a): x=1 return a+x print(x) print(f(16))</pre>
P3	
<pre>x=10 def f (a): return a+x print(x) print(f(16))</pre>	



Fonction lambda et expressions anonymes :

Une fonction lambda est une fonction anonyme qui peut prendre un nombre quelconque d'arguments, mais elle ne peut contenir qu'une seule expression.

```
#Syntax
fonction_lambda = lambda arguments: expression
#exemple
somme = lambda x, y: x + y
print(somme(5, 3)) # Affiche : 8
```



Fonction d'ordre supérieur

Prend une fonction en argument ou renvoie une fonction

Exemple :

```
def appliquer(fonction, x):  
    return fonction(x)  
  
def carre(n):  
    return n * n  
  
appliquer(carre, 5) # Résultat : 25
```





Fonctions d'ordre supérieur spéciales :

map() : Applique une fonction à tous les éléments d'un itérable.

```
def carre(x):  
    return x ** 2  
resultats = list(map(carre, [1, 2, 3, 4]))  
print(resultats) # Affiche : [1, 4, 9, 16]
```

● **filter()** : Filtre les éléments d'un itérable selon une fonction conditionnelle.

```
def est_pair(x):  
    return x % 2 == 0  
pairs = list(filter(est_pair, [1, 2, 3, 4, 5, 6]))  
print(pairs) # Affiche : [2, 4, 6]
```

● **reduce()** : Réduit une séquence à une seule valeur en appliquant une fonction cumulativement (nécessite d'importer depuis **functools**).

```
from functools import reduce  
somme = reduce(lambda x, y: x + y, [1, 2, 3, 4])  
print(somme) # Affiche : 10
```





Modules

- Les modules sont des programmes Python qui contiennent des fonctions qu'on peut les utiliser directement (on les appelle aussi bibliothèques ou *libraries*).



Exemple d'utilisation de modules

- Pour utiliser la fonction **f** du module **mod**, il faut importer cette fonction, ou le module entier avant l'utilisation de cette fonction. **Cela peut se faire de 3 manières :**



Importer le module

Pour utiliser la fonction **f** du module **mod**

```
>>>import mod  
>>>mod.f()
```



Importer le module comme un alias

- Certains noms de module sont longs, ou seront utilisés très souvent.
- On peut créer un **alias**, qui permettra d'appeler les fonctions de ce module.

```
>>> import mod as al
```

```
>>> al.f()
```

L'alias permet d'utiliser des noms plus courts dans le programme.



Importer la fonction du module

```
>>>from mod import f  
>>>f()
```



*** Pour importer toutes les fonctions**



Modules standards

- Des modules standards prêts à l'emploi sont livrés avec la distribution Python. Ex. **random**, **math**, **os**, etc.
- Liste complète : <https://docs.python.org/3/library/index.html>

Le module math



```
### Exponentielle et logarithme ###
e          # constante e, base de l'exponentielle
exp(x)     # exponentielle de x
log(x)     # logarithme népérien
log10(x)   # logarithme en base 10

### Fonctions trigonométriques ###
pi         # le nombre pi
cos(x)     # cosinus
sin(x)     # sinus
tan(x)     # tangente
acos(x)    # arccos
asin(x)    # arcsin
atan(x)    # arctan

### Autres fonctions ###
sqrt(x)    # racine carrée
factorial(x) # factorielle (pour x entier)
```



Le module random

`import : random`

`random.random()`

Renvoie le nombre aléatoire à virgule flottante dans la plage [0.0, 1.0].

`random.randint(a, b)`

Renvoie un entier aléatoire N tel que $a \leq N \leq b$.



Création d'un module personnalisé

Pour créer un module, il suffit de programmer les fonctions qui le constituent dans un fichier portant le nom du **nom_module.py**.

Depuis un (autre) programme en Python, il suffit alors d'utiliser **import** pour pouvoir utiliser ces fonctions.



Exercices

1. Écrire une fonction cube qui retourne le cube de son argument.
2. Écrire une fonction P telle que P(X) renvoie la valeur du polynôme $X^3 - 7X + 4$
3. Écrire une fonction V_Sphere(R) qui calcule le volume d'une sphère de rayon R fourni en argument.
$$V = \frac{4\pi R^3}{3}$$
4. Ecrire une fonction Nb_Premier(n) qui prend en paramètre un entier n et vérifie si le nombre est premier ou non (retourne **True** si le nombre est premier et **False** si non).
5. En déduire les nombres premiers de 393 à 511